
Trabajo Práctico Final

Implementación de un Blog Personal
sobre Infraestructura Virtualizada

Universidad Tecnológica Nacional
Facultad Regional Tucumán

Alumno: Lizarraga Juan

DNI: 44658603

Legajo: 56664

Materia: Virtualización y Consolidación de Servidores

Docente: Carriles, Luis María

Año 2026

Índice

1	Introducción	2
1.1	Objetivos	2
2	Arquitectura de la Solución	2
2.1	Diagrama de topología	2
2.2	Stack tecnológico	3
3	Configuración de la Infraestructura	3
3.1	Creación de los contenedores	3
3.2	Configuración de red	3
4	Configuración de la Base de Datos	4
4.1	Instalación y ajuste de memoria	4
4.2	Control de acceso	5
4.3	Esquema de datos	5
5	Desarrollo de la Aplicación Web	5
5.1	Modelo de funcionamiento: muro público	6
5.2	Datos personales e imagen	6
5.3	Estructura de archivos	6
5.4	Diseño de la interfaz	6
5.5	Despliegue automatizado	6
6	Seguridad	7
7	Verificación de Acceso Externo	7
7.1	Contexto	7
7.2	Verificación mediante túnel SSH reverso	7
8	Verificación y Pruebas	8
8.1	Verificación de la base de datos	8
8.2	Verificación del servidor web	8
8.3	Verificación de la integración completa	8
9	Conclusiones	8

1. Introducción

El presente informe documenta el desarrollo e implementación de un servicio de **Blog Personal** desplegado sobre una infraestructura virtualizada basada en **Proxmox VE**. El trabajo cumple con las especificaciones de la consigna: exposición de datos personales, imagen del alumno, y la capacidad de actualizar la página principal agregando nuevas entradas de forma dinámica.

La solución se montó sobre dos contenedores **LXC** independientes —uno para la aplicación web y otro para la base de datos— siguiendo el esquema de direccionamiento, máscara, puerta de enlace y DNS indicados en la consigna. La infraestructura es accesible a través de la dirección `nap.frt.utn.edu.ar`.

1.1. Objetivos

- Implementar un blog personal funcional que muestre datos e imagen del alumno.
- Permitir la actualización dinámica de la página principal mediante la carga de nuevas entradas.
- Desplegar la solución sobre dos contenedores LXC en Proxmox, con la configuración de red especificada.
- Aplicar buenas prácticas de seguridad en el desarrollo de la aplicación web.
- Exponer el servicio a Internet pese al aislamiento de la red privada institucional.

2. Arquitectura de la Solución

La arquitectura se compone de dos contenedores LXC alojados en los nodos Proxmox de la institución, comunicados entre sí a través de la red privada `172.16.90.0/24`. El usuario accede vía Internet, atravesando el router y el switch de la infraestructura hasta alcanzar el contenedor del blog.

2.1. Diagrama de topología

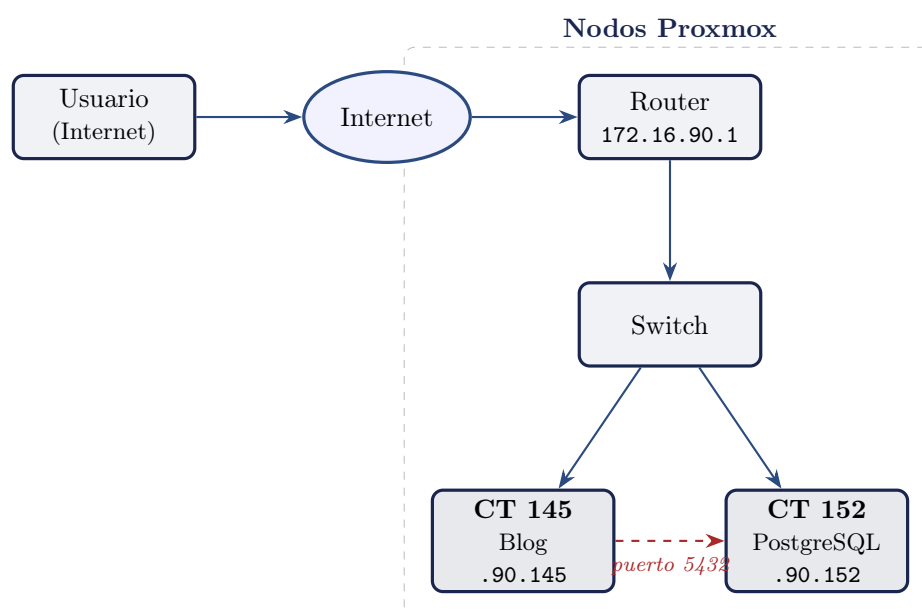


Figura 1: Topología de red: acceso del usuario y comunicación entre contenedores.

2.2. Stack tecnológico

La elección del stack estuvo condicionada por la principal restricción de la infraestructura: **128 MB de RAM por contenedor**. Esto descartó opciones pesadas (como un stack basado en JVM o frameworks PHP completos) en favor de componentes livianos.

Componente	Tecnología elegida
Servidor web	nginx 1.22
Procesador PHP	PHP 8.2 (FPM, pool <i>ondemand</i>)
Lenguaje de la aplicación	PHP 8.2 (sin framework)
Base de datos	PostgreSQL 15
Conector de base de datos	PDO_pgsql
Sistema operativo (contenedores)	Debian 12 (Bookworm)
Virtualización	Proxmox VE – Contenedores LXC
Exposición externa	Túnel SSH reverso (Pinggy)

Tabla 1: Componentes del stack tecnológico.

Justificación de decisiones:

- **nginx en lugar de Apache:** menor consumo de memoria, fundamental en 128 MB.
- **PHP plano (sin framework):** evita la sobrecarga de memoria de frameworks como Laravel o Symfony.
- **PostgreSQL:** motor robusto que, con un ajuste de parámetros adecuado, opera cómodamente dentro del límite de RAM.
- **Separación en dos contenedores:** aísla la capa de aplicación de la de datos, replicando una arquitectura real de producción.

3. Configuración de la Infraestructura

3.1. Creación de los contenedores

Se crearon dos contenedores LXC desde la interfaz web de Proxmox, utilizando la plantilla `debian-12-standard`. Ambos comparten las mismas especificaciones de recursos exigidas por la consigna.

El nombre de cada contenedor se construyó utilizando DNI como base, agregando el identificador correspondiente (A para el blog, DB para la base de datos).

3.2. Configuración de red

La dirección IP de cada contenedor se construyó tomando el CT ID asignado por Proxmox como último octeto, sobre la red `172.16.90.0/24`, conforme a la especificación de la consigna.

Parámetro	Contenedor Blog	Contenedor DB
CT ID	145	152
Hostname	44658603A	44658603DB
Memoria RAM	128 MB	128 MB
Swap	256 MB	256 MB
Procesadores	1 vCPU	1 vCPU
Almacenamiento	8 GB	8 GB
Plantilla	Debian 12	Debian 12

Tabla 2: Especificaciones de los contenedores.

Parámetro	Blog (CT 145)	DB (CT 152)
Dirección IP	172.16.90.145	172.16.90.152
Máscara de subred	255.255.255.0 (/24)	255.255.255.0 (/24)
Puerta de enlace	172.16.90.1	172.16.90.1
Servidor DNS	172.16.90.1	172.16.90.1
Bridge	vmbr0	vmbr0

Tabla 3: Configuración de red de ambos contenedores.

La configuración de red en contenedores LXC es gestionada por Proxmox a partir de los datos cargados en la interfaz, y se refleja en el archivo `/etc/resolv.conf` de cada contenedor. La verificación confirmó la correcta asignación del servidor DNS:

```
root@44658603A:~# cat /etc/resolv.conf
# --- BEGIN PVE ---
search alumnos
nameserver 172.16.90.1
# --- END PVE ---
```

4. Configuración de la Base de Datos

Sobre el contenedor 152 se instaló y configuró PostgreSQL 15.

4.1. Instalación y ajuste de memoria

Tras instalar el paquete `postgresql`, se ajustaron los parámetros del archivo `postgresql.conf` para operar dentro del límite de 128 MB de RAM. Los valores fueron conservadores para evitar que el proceso fuera terminado por falta de memoria (OOM).

```
listen_addresses = 'localhost,172.16.90.152'
shared_buffers = 24MB
work_mem = 1MB
maintenance_work_mem = 8MB
max_connections = 10
effective_cache_size = 48MB
```

Parámetros de `/etc/postgresql/15/main/postgresql.conf`

4.2. Control de acceso

En el archivo `pg_hba.conf` se restringió el acceso a la base de datos para que únicamente el contenedor del blog pueda conectarse, utilizando autenticación `scram-sha-256`:

```
host      blogdb      bloguser      172.16.90.145/32      scram-sha-256
```

Línea agregada a `pg_hba.conf`

Esta regla implementa el principio de mínimo privilegio a nivel de red: la base de datos solo acepta conexiones del host específico que las necesita.

4.3. Esquema de datos

Se creó la base de datos `blogdb` y el usuario `bloguser`. La tabla principal es `posts`, que almacena las entradas del blog (la actualización dinámica que exige la consigna).

```
CREATE USER bloguser WITH PASSWORD '*****';
CREATE DATABASE blogdb OWNER bloguser;
```

Creación del usuario y la base de datos

```
CREATE TABLE posts (
    id          SERIAL PRIMARY KEY,
    titulo      VARCHAR(200) NOT NULL,
    contenido   TEXT NOT NULL,
    fecha       TIMESTAMP DEFAULT NOW()
);
GRANT ALL PRIVILEGES ON TABLE posts TO bloguser;
GRANT USAGE, SELECT ON SEQUENCE posts_id_seq TO bloguser;
```

Tabla de entradas del blog

Nota: durante el desarrollo se evaluó una tabla `perfil` para almacenar los datos personales de forma editable. En la versión final se optó por mantener los datos personales y la foto de perfil como contenido estático en la configuración de la aplicación, por lo que dicha tabla quedó en desuso. Los datos del perfil no cambian, de modo que no se justificaba el costo de una consulta adicional a la base en cada carga de la página.

5. Desarrollo de la Aplicación Web

La aplicación se desarrolló en PHP 8.2 sin framework, priorizando un código claro y seguro. Se desplegó en el contenedor 145, en el directorio `/var/www/blog`.

5.1. Modelo de funcionamiento: muro público

La consigna exige que la página principal pueda *actualizarse agregando un nuevo dato*. Se resolvió este requisito mediante un modelo de **muro público**: la página principal presenta un formulario, visible para todo visitante, que permite publicar una nueva entrada. Al enviarse, la entrada se inserta en la base de datos y aparece de inmediato en el listado, ordenada de la más reciente a la más antigua.

Esta decisión de diseño convierte al blog en un espacio colaborativo. Para evitar que la apertura del formulario comprometa la seguridad, se mantuvieron de forma estricta las defensas de la capa de aplicación (sentencias preparadas, escape de salida, token CSRF y validación de longitud), descritas en la sección de Seguridad. La carga de entradas implementa el patrón *POST-Redirect-GET*, de modo que al recargar la página no se reenvía la última entrada.

5.2. Datos personales e imagen

Los datos personales (nombre, rol, descripción, carrera y universidad) y la foto de perfil se definen de forma estática en la configuración de la aplicación y en el directorio `assets/`.

5.3. Estructura de archivos

Archivo	Responsabilidad
<code>config.php</code>	Credenciales de la base de datos y datos personales estáticos.
<code>db.php</code>	Conexión PDO (singleton) a PostgreSQL.
<code>csrf.php</code>	Generación y verificación de tokens CSRF.
<code>index.php</code>	Página única: perfil, imagen, formulario de carga y listado de entradas.
<code>style.css</code>	Estilos de la interfaz (tema oscuro).
<code>assets/perfil.jpg</code>	Foto de perfil (estática).
<code>assets/fonts/</code>	Tipografía Inter alojada localmente.
<code>install.sh</code>	Script de despliegue automatizado.

Tabla 4: Estructura del proyecto.

5.4. Diseño de la interfaz

La interfaz se diseñó con un tema oscuro de aspecto profesional, con una sección de presentación (*hero*) que incluye la fotografía circular, nombre, rol y descripción, seguida del formulario de publicación y del listado de entradas en formato de tarjetas. La tipografía **Inter** se aloja localmente en el propio contenedor (no se carga desde un CDN externo), de modo que la aplicación es autocontenida y no depende de servicios de terceros para renderizar correctamente.

5.5. Despliegue automatizado

El script `install.sh` automatiza la instalación de paquetes, la copia de archivos a `/var/www/blog`, la generación de la configuración de nginx y del pool de PHP-FPM, el ajuste de permisos y el reinicio de los servicios. El pool de PHP-FPM se configuró en modo **ondemand** con un máximo de 3 procesos hijos, y nginx con un único proceso de trabajo, en línea con la restricción de RAM. La transferencia del código al contenedor se realizó mediante un repositorio Git, aprovechando que los contenedores cuentan con salida a Internet.

6. Seguridad

Pese a tratarse de un muro de publicación abierta, se aplicaron las siguientes medidas de seguridad en la capa de aplicación y de datos, verificables durante la defensa:

Amenaza	Mitigación implementada
Inyección SQL	Uso exclusivo de sentencias preparadas (PDO con <code>EMULATE_PREPARES=false</code>). Sin concatenación de cadenas en consultas.
Cross-Site Scripting (XSS)	Escape de toda salida con <code>htmlspecialchars()</code> con <code>ENT_QUOTES</code> y codificación UTF-8.
Cross-Site Request Forgery	Token CSRF en el formulario de publicación, verificado en el servidor antes de insertar.
Entradas inválidas	Validación del lado del servidor: campos no vacíos y límites de longitud (título 200, contenido 5000 caracteres).
Acceso a base de datos	Restricción por IP en <code>pg_hba.conf</code> : solo el contenedor del blog puede conectarse.
Aislamiento de datos	La base de datos no expone su puerto a Internet; solo es alcanzable desde la red privada por el contenedor de la aplicación.

Tabla 5: Medidas de seguridad implementadas.

7. Verificación de Acceso Externo

7.1. Contexto

La red `172.16.90.0/24` es una red privada interna de la institución. La exposición pública del blog a través de `nap.frt.utn.edu.ar` es gestionada por la cátedra mediante la configuración del proxy inverso en el borde de la infraestructura institucional, que redirige las peticiones entrantes hacia el contenedor correspondiente (`172.16.90.145:80`).

7.2. Verificación mediante túnel SSH reverso

Durante el desarrollo, con el fin de **verificar que la aplicación era correctamente accesible desde una red externa** antes de la instancia de defensa, se utilizó el servicio **Pinggy** para establecer un túnel SSH reverso temporal. Los contenedores disponen de salida a Internet, lo que permitió iniciar una conexión saliente cifrada desde el contenedor 145 hacia el servidor de Pinggy. El comando ejecutado fue:

```
ssh -p 443 -R 0:localhost:80 a.pinggy.io
```

Listing 1: Túnel reverso para verificación de acceso externo

Al enrutar la conexión por el puerto **443 (HTTPS)**, el tráfico es interpretado como tráfico web saliente legítimo. El servicio entregó una URL pública temporal con certificado TLS, desde la cual se comprobó que el blog cargaba correctamente desde una red domiciliaria, validando así el funcionamiento completo de la cadena: Internet → nginx → PHP-FPM → PostgreSQL.

Esta herramienta se utilizó exclusivamente como **mecanismo de verificación durante el desarrollo**. El acceso definitivo y permanente al blog se realiza a través de la infraestructura de la NAP de la institución, accesible vía `nap.frt.utn.edu.ar`.

8. Verificación y Pruebas

8.1. Verificación de la base de datos

Se confirmó la existencia de la tabla de entradas en la base `blogdb`:

```
blogdb=# \dt
          List of relations
 Schema | Name   | Type  | Owner
-----+-----+-----+-----
 public | posts  | table | postgres
```

8.2. Verificación del servidor web

La respuesta de cabeceras HTTP confirmó que nginx sirve la aplicación correctamente:

```
root@44658603A:~# curl -I http://172.16.90.145/
HTTP/1.1 200 OK
Server: nginx/1.22.1
Content-Type: text/html
...
```

8.3. Verificación de la integración completa

La prueba definitiva consistió en solicitar la página principal completa. Su correcta generación demuestra que la cadena completa funciona: nginx recibe la petición, PHP-FPM ejecuta `index.php`, este se conecta a PostgreSQL en el contenedor 152, consulta las entradas y devuelve el HTML resultante. Esto valida la comunicación entre ambos contenedores a través de la red privada. Asimismo, se verificó la publicación de una entrada de prueba desde el formulario, comprobando su aparición inmediata en el listado.

9. Conclusiones

Se implementó exitosamente un servicio de blog personal sobre una infraestructura de dos contenedores LXC en Proxmox, cumpliendo con las especificaciones de la consigna: datos personales, imagen del alumno, actualización dinámica de la página principal, configuración de red exigida (direccionamiento basado en el CT ID, máscara, gateway y DNS) y nomenclatura de contenedores.

El principal desafío técnico fue operar dentro de la restricción de 128 MB de RAM por contenedor, lo que guió la elección de un stack liviano (nginx + PHP-FPM + PostgreSQL ajustado) y el descarte de alternativas más pesadas. La separación de la aplicación y la base de datos en contenedores distintos, comunicados por una red privada con control de acceso, reproduce una arquitectura representativa de entornos reales de producción. La correcta accesibilidad del blog desde redes externas fue verificada durante el desarrollo mediante un túnel SSH reverso, confirmando el funcionamiento integral de la cadena de servicios.

Acceso público: el blog queda publicado y accesible a través de `nap.frt.utn.edu.ar/44658603` mediante la configuración de proxy inverso gestionada por la cátedra, que redirige las peticiones hacia el contenedor 172.16.90.145 en el puerto 80.